

Project Proposal Template

Project: Hospital Smart Queue System

Objective: To design and develop a comprehensive smart queue and appointment management system for hospitals. The system aims to streamline patient flow, reduce waiting times, and provide a centralized platform for admins, receptionists, doctors, and patients to manage appointments efficiently. The system will replace manual queue processes with an automated, real-time digital solution.

Description:

The Hospital Smart Queue System is a web-based application with four distinct user dashboards tailored to specific roles:

Admin Dashboard: The admin has full control over staff management. They can add, edit, or remove other admins, doctors, and receptionists by assigning them email credentials and passwords. The admin can view all user data, manage hospital departments, and access an analytics section displaying weekly appointment statistics and patient attendance rates.

Receptionist Dashboard: The receptionist manages the daily operational schedule. They have the authority to create new appointments for patients (walk-ins or phone bookings), modify existing appointment details (time, doctor, department), and cancel appointments when necessary. They have a clear view of the daily schedule across all departments.

Patient Dashboard: Patients can browse all available hospital departments and view the list of doctors within each department. They can book appointments based on doctor availability, view their complete appointment history (upcoming and past), and receive real-time notifications reminding them of approaching appointment times.

Doctor Dashboard: Doctors can view their personalized daily schedule showing all patient appointments. After a consultation, the doctor can mark the appointment as "Completed," which

updates the system's analytics. Doctors also have the ability to set "Block Time" intervals to indicate periods they are unavailable for bookings (e.g., surgery, break, meeting).

Tools & Design:

Vs code

GitHub

Design: Figma, Stitch AI

Technologies to Use:

Frontend: React.js, Tailwind CSS

Backend: Node.js, Express.js

Database: MongoDB

Real-time Features: Socket.IO

Authentication: JSON Web Tokens (JWT)

Team Members & Responsibilities:

- 1- Yassin Mostafa Wahb [Team Leader] – Authentication Pages (Sign In / Register) & Admin Page
- 2- Yousef Tamer Moharam – Admin Page (Staff Management & Analytics Sections)
- 3- Abdelrahman Ahmed Amir – Patient Page (Department Browsing, Booking, Appointments View)

- 4- Kevin Akram Nader – Doctor Page (Daily Schedule, Mark Complete, Block Time) 5-
John Ehab Samir – Receptionist Page (Create/Modify/Cancel Appointments, Daily
Schedule)
-

Week 1: *[Tasks Description With Team Member Responsibility]*

- Yassin: Design and implement Sign In & Register UI (React + Tailwind). Build corresponding backend API endpoints (Node/Express) for user registration and login with JWT authentication. Share backend logic with Yousef for Admin integration.
- Yassin & Yousef: Collaborate on Admin Page routing and layout structure. Yassin handles authentication redirect logic to Admin dashboard. Yousef begins building Admin dashboard UI skeleton.
- Yousef: Begin building backend API endpoints for Admin functionality (fetch all users, add staff). Connect to MongoDB.
- Abdelrahman: Set up Patient Page frontend routing. Build backend API endpoints for fetching departments and doctors list.
- Kevin: Set up Doctor Page frontend routing. Build backend API endpoints for fetching doctor's daily schedule.
- John: Set up Receptionist Page frontend routing. Build backend API endpoints for fetching daily schedule and appointment CRUD operations.
- All Members: Initialize GitHub repository with branch strategy (each member works on their own feature branch). Connect backend server to MongoDB Atlas.

Deliverables:

- Fully functional Sign In / Register system (Frontend + Backend) by Yassin
 - GitHub Repository with individual branches created
 - MongoDB connected and User Schema defined
 - Each member's basic API structure initialized
-

Week 2: *[Tasks Description With Team Member Responsibility]*

- Yousef: Implement Staff Management in the Admin Panel (Forms to add Doctors and Receptionists, display list of existing users), Implement Departments Management (Add/View Departments) in Admin Panel.
- Yassin: Assist Yousef in connecting Admin forms to backend API endpoints and testing.
- Abdelrahman: Design and build the Patient Home Page (Display list of Departments as cards), Build the Doctors List Page (Filter doctors by selected Department, show availability).
- John: Begin building the backend API logic for Creating Appointments (used by Receptionist).

Deliverables:

- Admin can successfully add a new Doctor/Receptionist via the dashboard.
- Patient can browse Departments and view Doctors by department.
- Backend API endpoints for /departments, /doctors, and /users completed and tested.

Week 3: *[Tasks Description With Team Member Responsibility]*

- Abdelrahman: Implement Appointment Booking Form (Select Date, Time Slot, Doctor) and save to database, build "My Appointments" page for patient showing upcoming and past bookings.
- John: Complete Receptionist Dashboard with full CRUD (Create, Read, Update, Cancel) functionality for any patient appointment, Build receptionist view for daily schedule across all doctors.
- Kevin: Build the Doctor Dashboard to fetch and display Today's Appointments for the logged-in doctor, implement "Mark as Completed" button functionality with database update.
- All Members: Begin Socket.IO integration for real-time notifications.

Deliverables:

- Patient can successfully book an appointment with selected doctor.
- Receptionist can create, modify, and cancel any appointment.
- Doctor can see their live schedule and mark a patient as "Completed".
- Real-time updates reflect across all relevant dashboards.

Week 4: *[Tasks Description With Team Member Responsibility]*

- Yousef: Implement Admin Analytics Dashboard (Charts using Chart.js or Recharts) showing Weekly Appointments vs. Attendance Rate.
- Kevin: Implement Doctor "Block Time" UI and logic (Calendar selection to mark unavailable slots that prevent patient bookings).
- John: Finalize Receptionist View filtering and search functionality for daily schedule.
- Abdelrahman: Implement Patient Notifications (In-app alert or toast message) for approaching appointments using Socket.IO.
- Yassin: Finalize overall authentication flow and session management across all roles.
- All Members: Full system integration testing (End-to-End user journey: Admin creates Doctor -> Patient books -> Receptionist modifies -> Doctor completes).
- All Members: Bug fixing and final deployment preparation.

Deliverables:

- Fully functional Admin Analytics chart displaying weekly data.
- Doctor Block Time preventing patient bookings during blocked periods.
- Patient receives notification reminder 1 hour before appointment.
- Complete end-to-end system tested and functional.
- Final Project Demo ready for presentation.

KPIs and Risk Assessment & Mitigation Plan

Key Performance Indicators (KPIs)

System Uptime: Target $\geq 99\%$

Average Response Time: ≤ 2 seconds

Appointment Booking Success Rate: $\geq 95\%$

No-show Rate Reduction: $\geq 30\%$

User Adoption Rate: $\geq 80\%$ of staff actively using system

Real-time Update Latency: ≤ 1 second

Error Rate: $\leq 2\%$ failed requests

Average Waiting Time Reduction: $\geq 25\%$

Testing Plan

The project will utilize Jest as the primary testing framework for both frontend and backend unit testing. Jest will be used to test individual components, API endpoints, and utility functions to ensure code reliability and catch bugs early in the development process.

Risk Assessment & Mitigation Plan

Authentication Security Risk

Risk: Unauthorized access due to weak authentication

Mitigation: Use JWT with expiration, hashing passwords, role-based access control

Double Booking

Risk: Two patients booking same time slot

Mitigation: Implement server-side validation and database constraints

System Downtime

Risk: Server crashes or hosting issues

Mitigation: Use reliable hosting and monitoring tools **Data**

Loss

Risk: Database failure or accidental deletion Mitigation:

Regular automated backups

Real-time Sync Failure

Risk: Socket.IO events not updating properly

Mitigation: Fallback to API polling and proper event handling

User Misuse

Risk: Receptionist or admin errors

Mitigation: Add validation and confirmation steps

Scalability Issues

Risk: System slows with many users

Mitigation: Optimize queries and use scalable architecture

Network Issues

Risk: Users lose connection

Mitigation: Retry mechanisms and offline handling